

Parallelization of K-Means Clustering using OpenMP - Lab 2

Angelos Gkertsos (03658)

Nikolaos Kalamaris (03768)

Contents

1	Introduction	2
1.1	K-Means Algorithm Overview	2
1.2	Motivation for Parallelization	2
1.3	OpenMP Approach	2
2	Methodology	2
2.1	System Environment	2
2.2	Input Data and Parameters	2
2.3	Correctness	2
2.4	Performance Measurement	2
2.5	Profiling	2
3	Profiling Results and Identification of Hotspots	3
4	Parallelization Strategies and Initial Performance	3
4.1	Strategy V1: Loop Parallelism with Atomic Synchronization (schedule(static))	3
4.1.1	Method	3
4.1.2	V1 (Static) Performance Results	3
4.1.3	V1 (Static) Initial Analysis	3
4.2	Strategy V2: Separate Parallel Regions for Both Hotspots	4
4.2.1	Method	4
4.2.2	V2 Performance Results	4
4.2.3	V2 Analysis: Fork-Join Overhead	4
4.3	Strategy V3: Single Parallel Region with Private Accumulators and Parallel Reduction	4
4.3.1	Method	4
4.3.2	V3 Performance Results	5
4.3.3	V3 Analysis: Reduction Bottleneck	5
4.4	Comparison of Initial Strategies	5
5	Schedule Tuning for Optimal Strategy (V1)	6
5.1	Rationale for Tuning V1	6
5.2	Schedule Policies Overview	6
5.3	Tuning Results Comparison	6
5.4	Optimal Schedule Selection	7
6	Final Discussion	8
7	Conclusion	8

1 Introduction

1.1 K-Means Algorithm Overview

The K-Means algorithm partitions a dataset (n points, d dimensions) into k clusters through an iterative process. Each iteration involves assigning points to the nearest centroid and subsequently updating centroid positions based on the mean of assigned points, continuing until convergence.

1.2 Motivation for Parallelization

The core computation, particularly the assignment step ($O(n \times k \times d)$ per iteration), becomes a bottleneck for large datasets. Parallel computing offers a means to accelerate this process by distributing the workload across multiple processor cores.

1.3 OpenMP Approach

OpenMP utilizes compiler directives (`#pragma omp ...`) to enable shared-memory parallelism, often requiring minimal changes to sequential C/C++/Fortran code. Its focus on loop parallelization makes it suitable for algorithms like K-Means.

2 Methodology

2.1 System Environment

Hardware: csl-venus (10.64.82.60): 2 x Intel Xeon E5-2695 v3 CPUs (14 cores/CPU, 2-way SMT), 28 physical cores / 56 logical threads total. **Compiler:** Intel C Compiler (icx). **Flags:** `-O4 -DNDEBUG -I.` for sequential (`seq_main`); `-O4 -DNDEBUG -I. -qopenmp` for OpenMP (`omp_main`).

2.2 Input Data and Parameters

Dataset: texture17695.bin ($n=17695$, $d=20$). **K-Means:** $k=2000$, $\text{threshold}=0.001$.

2.3 Correctness

To ensure correctness before benchmarking, we first generated a "ground truth" output file by running the sequential code with a small, verifiable input (color100.txt).

Then, for each new parallel strategy (V1, V2, and V3), we ran it using the exact same input. We used the diff command to compare the output file from the parallel version against the sequential "ground truth" file.

2.4 Performance Measurement

Timing: Execution time of the core K-Means computation was measured using functions provided in `wtime.c`, excluding file I/O time. The `-o` flag enabled this timing output. **Repetitions:** Each configuration was run 12 times. Mean and standard deviation are reported. **Speedup:** Calculated as $S_N = T_{\text{Sequential}}/T_{\text{Parallel}}(N)$, where $T_{\text{Sequential}}$ is the mean sequential time measured during the specific test run being analyzed.

2.5 Profiling

GNU Profiler (gprof) used on sequential code (compiled with `-pg`).

3 Profiling Results and Identification of Hotspots

Profiling the original sequential code using gprof consistently showed that nearly 100% of the execution time occurred within the `seq_kmeans` function. This confirmed it as the sole area requiring optimization.

The flat profile obtained from gprof is as follows:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self Ts/call	total Ts/call	name
100.00	3.45	3.45				seq_kmeans

Within `seq_kmeans`, the `do-while` loop performs the K-Means iterations. Analysis identified two key computational sections inside this loop:

- **Hotspot 1:** The primary loop (`for (i=0; i<numObjs; i++)`). This loop iterates through all data points, finds the nearest cluster for each (calling `find_nearest_cluster`), assigns membership, and accumulates sums and counts into the temporary `newClusterSize` and `newClusters` arrays. Its complexity per K-Means iteration is $O(n \cdot k \cdot d)$, where n =numObjs, k =numClusters, d =numCoords. This loop dominates the runtime.
- **Hotspot 2:** The secondary loop (`for (i=0; i<numClusters; i++)`). This loop calculates the new centroid positions based on the sums accumulated in Hotspot 1 and resets the temporary arrays. Its complexity per K-Means iteration is $O(k \cdot d)$.

Profiling indicated that loops in the beginning of `int omp_kmeans()` and outside of the `do while` loop, like `for (i=0; i<numObjs; i++) membership[i] = -1;` and `for (i=1; i<numClusters; i++) newClusters[i] = newClusters[i-1] + numCoords;`, constitute “cold code”—their contribution to the total execution time is negligible compared to Hotspots 1 and 2. Therefore, parallelization efforts were correctly focused on the main iterative hotspots.

4 Parallelization Strategies and Initial Performance

4.1 Strategy V1: Loop Parallelism with Atomic Synchronization (`schedule(static)`)

4.1.1 Method

This first parallelization strategy targeted the main computational bottleneck (Hotspot 1) using basic OpenMP loop parallelism.

Parallel Loop: An `#pragma omp parallel for` directive was applied to the `for (i=0; i<numObjs; i++)` loop.

Synchronization: `#pragma omp atomic` was used for thread-safe updates to shared `newClusterSize` and `newClusters` arrays. `reduction(+:delta)` handled the `delta` variable summation.

Schedule: The default `schedule(static)` was used initially.

Scope: Hotspot 2 remained sequential.

4.1.2 V1 (Static) Performance Results

Note: Speedup calculated relative to the sequential time measured in this specific run (3.5925s).

4.1.3 V1 (Static) Initial Analysis

The initial V1 implementation showed significant speedup (up to 18.2x). The 1-thread case demonstrated the expected OpenMP overhead, running slightly slower than the pure sequential code (0.98x speedup). Scalability was reasonable but clearly limited, especially at higher thread counts. The poor performance with static scheduling, particularly the instability observed at 28 threads in some runs, suggested potential load imbalance among threads, motivating schedule tuning later.

Table 1: Performance of V1 with schedule(static)

Threads	Mean Time (s)	StdDev (s)	Speedup
Seq (Baseline)	3.5925	0.057	1.00x
1 (OMP)	3.6592	0.165	0.98x
4 (OMP)	1.0853	0.039	3.31x
8 (OMP)	0.6296	0.026	5.71x
14 (OMP)	0.3711	0.018	9.68x
28 (OMP)	0.2530	0.055	14.20x
56 (OMP)	0.1974	0.022	18.20x

4.2 Strategy V2: Separate Parallel Regions for Both Hotspots

4.2.1 Method

This strategy attempted to parallelize the remaining sequential work (Hotspot 2) by adding a second, independent `#pragma omp parallel` for directive within the `do-while` loop.

4.2.2 V2 Performance Results

Table 2: Performance of V2 (Separate Parallel Regions)

Threads	Mean Time (s)	StdDev (s)	Speedup
Seq (Baseline)	3.5888	0.1601	1.00x
1 (OMP)	3.8119	0.2081	0.94x
4 (OMP)	1.1271	0.0392	3.18x
8 (OMP)	0.6178	0.0270	5.81x
14 (OMP)	0.3752	0.0189	9.56x
28 (OMP)	0.2321	0.0506	15.46x
56 (OMP)	0.2039	0.0211	17.66x

4.2.3 V2 Analysis: Fork-Join Overhead

Comparing V2 results to V1 (static), V2 was generally slower, especially at high thread counts (max speedup $\sim 17.7x$ vs $\sim 18.2x$). This performance degradation was attributed to the excessive fork-join overhead. Creating and synchronizing two separate thread teams within each `do-while` iteration imposed a significant cost that outweighed the negligible benefit of parallelizing the fast Hotspot 2. This indicated V1 was a better structural approach.

4.3 Strategy V3: Single Parallel Region with Private Accumulators and Parallel Reduction

This advanced strategy aimed to overcome both V1’s atomic contention and V2’s fork-join overhead.

4.3.1 Method

Single Parallel Region: A single `#pragma omp parallel` was placed outside the `do-while` loop.

Private Accumulators: Each thread allocated and used private arrays (`local_...`) to store cluster sums and counts during Hotspot 1 (parallelized via `#pragma omp for`, no atomics).

Parallel Reduction: After Hotspot 1 within each iteration, a reduction step combined the private arrays into the global arrays. This was implemented using a second `#pragma omp for` that parallelized the reduction across clusters. Each thread processed a subset of clusters, summing contributions from all threads’ private arrays for those clusters.

Synchronization: `barrier` directives ensured correct ordering between computation, reduction, and the parallelized Hotspot 2 update step (also using `#pragma omp for`).

4.3.2 V3 Performance Results

Table 3: Performance of V3 (Single Region, Parallel Reduction)

Threads	Mean Time (s)	StdDev (s)	Speedup (vs $T_{\text{Seq}}=3.6176\text{s}$)
Seq (Baseline)	3.6176	0.1190	1.00x
1 (OMP)	3.9366	0.1264	0.92x
4 (OMP)	1.1994	0.0558	3.02x
8 (OMP)	0.6582	0.0336	5.50x
14 (OMP)	0.4046	0.0157	8.94x
28 (OMP)	0.2606	0.0469	13.88x
56 (OMP)	0.2195	0.0163	16.41x

4.3.3 V3 Analysis: Reduction Bottleneck

Strategy V3 performed significantly worse than V1, achieving only $\sim 16.4\text{x}$ speedup. The primary reason was the manual parallel reduction step becoming a new bottleneck. Although parallelized, this step involved complex synchronization (barriers) and potentially inefficient memory access patterns (reading from all threads' private data), introducing significant overhead. Furthermore, zeroing the private arrays each iteration added cost.

A contributing factor relates to the problem size relative to thread count and clusters. The rule of thumb:

$$\begin{aligned}
N &= \text{numObjs} \\
C &= \text{numClusters} \\
D &= \text{numCoords} \\
T &= \text{numThreads}
\end{aligned}$$

Rough per-iteration work:

$$\text{V1 atomics} \approx N \times D \text{ atomic adds (plus atomic increments for sizes)}$$

$$\text{V3 reduction/zero} \approx T \times C \times D \text{ (zero)} + T \times C \times D \text{ (reduce)} = \sim 2 \times T \times C \times D$$

So V3 wins when:

$$N \times D \gg T \times C \times D \quad \Rightarrow \quad N \gg T \times C$$

i.e., when the number of objects is much larger than $T \times C$. If this is not true in your run, V1 is expected to be faster.. In our case, $N=17695$ while $T \times C = 56 \times 2000 = 112000$. Since N is much smaller than $T \times C$, the overhead of managing and reducing the large private arrays (size related to $T \times C$) likely outweighs the benefit gained by avoiding atomics (cost related to $N \times d$).

4.4 Comparison of Initial Strategies

Comparing the peak performance (at 56 threads) of the initial strategies:

- V1 (static): $\sim 18.2\text{x}$ speedup
- V2: $\sim 17.7\text{x}$ speedup
- V3: $\sim 16.4\text{x}$ speedup

The simplest approach, V1, demonstrated the best potential, although hindered by static scheduling. Therefore, V1 was selected for further tuning.

5 Schedule Tuning for Optimal Strategy (V1)

5.1 Rationale for Tuning V1

The V1 strategy (parallel Hotspot 1 with atomics, sequential Hotspot 2) offered the best baseline performance among the tested structures. However, its initial static scheduling results suggested potential load imbalance. Tuning the `schedule` clause aimed to mitigate this imbalance and maximize V1’s performance.

5.2 Schedule Policies Overview

- **static**: Low overhead, pre-assigns iterations. Prone to imbalance if iteration costs vary.
- **dynamic[, chunk]**: High overhead, assigns chunks on demand. Excellent load balancing. Small chunks increase overhead but improve balance.
- **guided[, chunk]**: Adaptive compromise. Starts with large chunks (low overhead), reduces chunk size towards the end (improves balance).

5.3 Tuning Results Comparison

Tables with the tuning results of V1 strategy using different schedule policies.

Table 4: V1 STATIC Schedule Results

Threads	Mean (s)	StdDev (s)	Speedup
Sequential	3.5925	0.057	1.00x
1 (OMP)	3.6592	0.165	0.95x
4 (OMP)	1.0853	0.039	3.31x
8 (OMP)	0.6296	0.026	5.71x
14 (OMP)	0.3711	0.018	9.68x
28 (OMP)	0.2530	0.055	14.20x
56 (OMP)	0.1974	0.022	18.20x

Table 5: V1 DYNAMIC Schedule Results

Threads	Mean (s)	StdDev (s)	Speedup
Sequential	3.583	0.0984	1.00x
1 (OMP)	3.6652	0.1246	0.98x
4 (OMP)	1.0163	0.0305	3.52x
8 (OMP)	0.5556	0.0125	6.45x
14 (OMP)	0.3377	0.0076	10.61x
28 (OMP)	0.1797	0.0039	19.94x
56 (OMP)	0.1756	0.0068	20.40x

Table 6: V1 DYNAMIC,1 Schedule Results

Threads	Mean (s)	StdDev (s)	Speedup
Sequential	3.5829	0.1197	1.00x
1 (OMP)	3.619	0.1264	0.99x
4 (OMP)	1.036	0.0384	3.46x
8 (OMP)	0.5598	0.0125	6.40x
14 (OMP)	0.3341	0.0076	10.72x
28 (OMP)	0.1791	0.0039	20.00x
56 (OMP)	0.1764	0.0051	20.31x

Table 7: V1 DYNAMIC,64 Schedule Results

Threads	Mean (s)	StdDev (s)	Speedup
Sequential	3.5884	0.1228	1.00x
1 (OMP)	3.6167	0.1292	0.99x
4 (OMP)	1.0269	0.027	3.50x
8 (OMP)	0.5619	0.0163	6.39x
14 (OMP)	0.3392	0.0076	10.58x
28 (OMP)	0.1834	0.0055	19.57x
56 (OMP)	0.1786	0.0059	20.09x

Table 8: V1 GUIDED Schedule Results

Threads	Mean (s)	StdDev (s)	Speedup
Sequential	3.6019	0.0886	1.00x
1 (OMP)	3.5753	0.1278	1.01x
4 (OMP)	1.0203	0.0211	3.53x
8 (OMP)	0.5635	0.0121	6.39x
14 (OMP)	0.3367	0.0075	10.70x
28 (OMP)	0.1800	0.0045	20.01x
56 (OMP)	0.1759	0.0053	20.48x

Table 9: V1 GUIDED,64 Schedule Results

Threads	Mean (s)	StdDev (s)	Speedup
Sequential	3.5479	0.125	1.00x
1 (OMP)	3.5901	0.0988	0.99x
4 (OMP)	1.0243	0.0229	3.46x
8 (OMP)	0.5664	0.0125	6.27x
14 (OMP)	0.3409	0.0084	10.41x
28 (OMP)	0.1893	0.0051	18.74x
56 (OMP)	0.1842	0.0058	19.26x

Table 10: V1 Schedule Policy Comparison @ 56 Threads (Speedup vs Each Schedule’s Sequential Time)

Schedule	Mean Time (s) @ 56 Thr	Speedup (vs own T_{Seq})
static	0.1974	$3.5925/0.1974 = 18.20x$
dynamic	0.1756	$3.583/0.1756 = 20.40x$
dynamic, 1	0.1764	$3.5829/0.1764 = 20.31x$
dynamic, 64	0.1786	$3.5884/0.1786 = 20.09x$
guided	0.1759	$3.6019/0.1759 = 20.48x$
guided, 64	0.1842	$3.5479/0.1842 = 19.26x$

5.4 Optimal Schedule Selection

The results clearly show that both **dynamic** (default chunk ≈ 1) and **guided** (default chunk) substantially outperform **static**. They achieve nearly identical peak performance, significantly improving speedup from $\sim 18.2x$ to $\sim 20.5x$. Larger chunk sizes (64) slightly degraded performance. Between **dynamic** and **guided**, **guided** offers a good theoretical balance and achieved the marginally highest measured speedup, making `schedule(guided)` the final choice for the optimized V1 implementation.

6 Final Discussion

The final performance of the optimized V1 implementation exhibits several key characteristics:

- **OpenMP Overhead:** The 1-thread parallel execution is slightly slower than the pure sequential code (0.98x speedup), confirming the small but measurable overhead introduced by the OpenMP framework.
- **Scalability:** The application demonstrates strong, albeit sub-linear, scaling up to the 28 physical cores, achieving a remarkable $\sim 20x$ speedup.
- **Limiting Factor (Atomic Contention):** The deviation from ideal linear speedup (decreasing efficiency) is primarily due to contention on the atomic operations. As more threads concurrently attempt to update shared cluster statistics, serialization increases, limiting parallel gains.
- **SMT (Hyperthreading) Impact:** Utilizing all 56 logical threads provides only a minimal additional speedup (from $\sim 20.0x$ to $\sim 20.5x$) compared to using only the 28 physical cores. This indicates that SMT is largely ineffective for this workload, likely because the frequent atomic operations hinder the concurrent execution of logical threads sharing a physical core. Efficiency drops significantly when SMT is fully utilized.
- **Schedule Importance:** The selection of `schedule(guided)` was crucial, providing a substantial improvement over `static` by ensuring effective load balancing among threads.

7 Conclusion

The parallelization of the sequential K-Means algorithm using OpenMP successfully yielded significant performance improvements. Profiling identified the main data point assignment loop as the critical performance bottleneck. A stepwise evaluation compared three distinct strategies: V1 (loop parallelism with atomics), V2 (naive parallelization of a secondary loop, highlighting fork-join overhead), and V3 (single parallel region with private arrays, revealing reduction bottlenecks).

The most effective strategy was V1, which, when combined with optimized load balancing using `schedule(guided)`, achieved a maximum speedup of approximately 20.5x on the 56-thread target system. While theoretically appealing, the more complex V3 strategy proved less efficient due to the high cost of its manual reduction step for this specific problem size.

The final results demonstrate the successful application of OpenMP for accelerating K-Means, achieving substantial speedup through careful targeting of the main hotspot and tuning of parallel execution parameters like scheduling. The analysis also underscores the practical performance limitations imposed by synchronization mechanisms (atomic contention) and architectural features (SMT effectiveness).